

# Software Requirements Specification

## Introduction

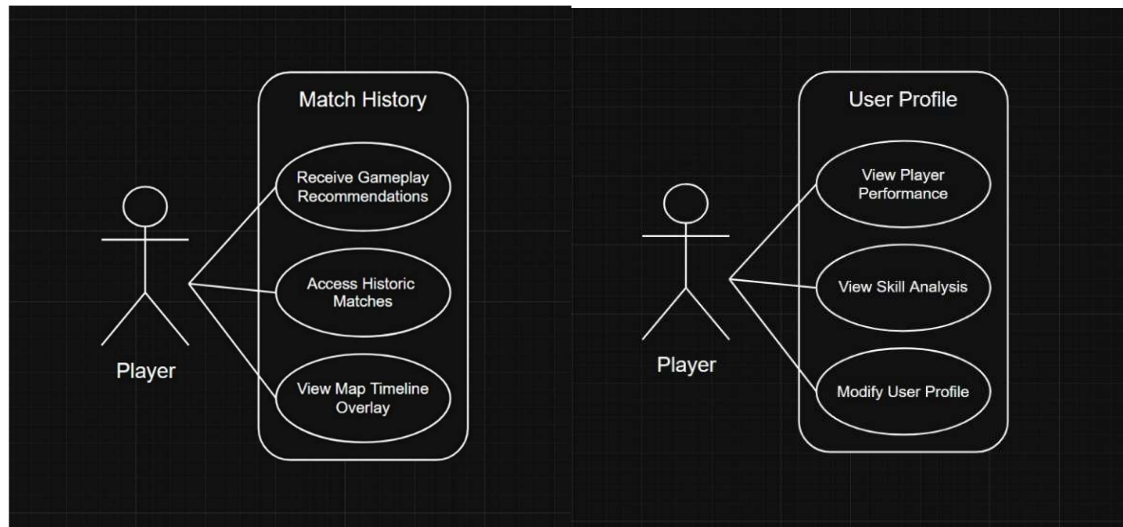
In competitive gaming, post-match analysis is often limited to video replays (VODs), which lack objective spatial data. Professional teams and analysts need a way to visualize positioning, rotations, and map control through coordinate-based data to identify patterns that are invisible to the naked eye.

Vantage Point will ingest game coordinate data, process spatial relationships, and provide a dashboard for analyzing player positioning and team density over the course of a match

## User Characteristics and User Stories

Our main user will be the players of League of Legends who wish to have more insight on their matches and use this information to improve their gameplay, at any level of skill. These players wish to be able to go back, and replay matches to understand more about the game flow and alternative paths and actions they could have taken.

| User Stories                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| "As a <b>player</b> , I want a breakdown for <b>performance over the last 5 matches</b> so that I can <b>see and understand my current skill level</b> ." |
| "As a <b>user</b> , I want to be able to <b>edit my profile information</b> so that I can <b>customize it</b> to myself."                                 |
| "As a <b>player</b> , I want to <b>access my previous matches</b> and their data/stats so that I can go back and <b>learn from them</b> ."                |
| "As a <b>player</b> , I want a <b>map overlay</b> of my matches so that I can <b>understand matches as a whole</b> ."                                     |
| "As a <b>player</b> , I want <b>match timelines</b> with plotted data and analysis so that I can <b>see what went wrong and when</b> ."                   |
| "As a <b>player</b> , I want <b>personalized gameplay recommendations</b> so that I can have <b>information to improve my skills</b> ."                   |



## Functional Requirements

---

### Front-End Layer

- Match History Dashboard
  - FR1.** A centralized interface displaying recent matches with basic stat summaries, serving as the entry point for deeper analysis.
- Raw Stat Visualization
  - FR2.** Visual representation of standard metrics (K/D/A, CS, KPM, etc.) alongside spatial data, bridging traditional stats with advanced positioning insights.

### ML and Data Processing Layer

- 2D Map Overlays
  - FR3-1.** Spatial Tracking Engine: Process (x, y) coordinate data for player deaths and kills across match timelines.
  - FR3-2.** Interactive Map Rendering: Create dynamic overlays of Summoner's Rift with plottable data points for visual clarity
  - FR3-3.** Spatial Delta Engine: Calculate vector differences between actual player coordinates and "Target" coordinates derived from pro-standard success states.
- Comparative Pro Analysis
  - FR4.** Overlay user positioning data against "ideal" benchmarks from high-ranking professional matches, providing aspirational targets.

### API and Backend Layer

- Automated Data Ingestion
  - FR5.** Integration with Riot's Match-v5 API to fetch comprehensive match histories and timelines, enabling complete and accurate datasets for analysis.

- User Authentication and Profiles

**FR6.** Secure login system with the ability for users to link their Riot Games IDs, ensuring personalized data access and analysis.

## API Service Contracts

---

### 1. Authentication Endpoints

#### 1.1 Register User

##### **POST /api/auth/register**

- Creates a new User Account

Request Body:

```
{
  "username": "Player1",
  "email": "player@example.com",
  "password": "SecurePass123"
}
```

Response Body:

```
{
  "message": "User registered successfully"
}
```

#### 1.2 Login User

##### **/api/auth/login**

- Authenticates a user and returns Cognito JWT tokens.

Request Body:

```
{
  "password": "securepassword123",
  "username": "Sn1per1"
}
```

Response Body:

```
{
  "AccessToken": "eyJraWQiOiJFMnk3eDc...",
  "ExpiresIn": 3600,
  "TokenType": "Bearer",
  "RefreshToken": "eyJjdHkiOiJKV1QiLCJ...",
  "IdToken": "eyJraWQiOiJsMmVHZ0g..."
}
```

#### 1.3 Confirm Email

##### **POST /api/auth/confirm**

- Verifies the user's email registration using the Cognito confirmation code.

Request Body:

```
{
  "username": "Sn1per1",
  "confirmation_code": "123456"
}
```

```
Response Body:
{
  "status": "success"
}
```

#### 1.4 Logout User

##### **/api/auth/logout**

- Globally invalidates the user's active session across all devices.
- Requires Authentication: Bearer <Access Token>

```
Response Body:
{
  "status": "success"
}
```

## 2. User Profile Endpoints

All endpoints in this section require an authentication header: Authorization: Bearer <Token>.

#### 2.1 Get Profile Summary

##### **GET /api/profile**

- Retrieves the authenticated user's profile details and a parsed aggregate performance summary.

```
Response Body:
{
  "puuid": "b0fc69dc-40a1-704a-5302-a6c936519de9",
  "username": "Sn1per1",
  "total_matches": 142,
  "player_summary": {
    "most_played_character": "Jinx",
    "common_mistakes": [
      "Low vision score",
      "Overextending late game"
    ],
    "avg_kda": "8.4 / 4.2 / 6.1",
    "win_rate": "54%"
  }
}
```

#### 2.2 Update Riot Key

##### **PUT /api/profile/riot-key**

- Temporarily updates the Riot API key configuration for the user's current session.

```
Request Body:
{
  "riot_api_key": "RGAPI-xxx..."
}
```

```
Response Body:
{
  "message": "Riot API Key updated successfully for this session.",
}
```

```
"user": "Sn1per1",
"status": "mock_verified"
}
```

### 2.3 Delete Account

#### **DELETE /api/profile**

- Places the authenticated account into a 30-day grace period queue before permanent deletion.

```
Response Body:
{
  "message": "Account marked for deletion. You have 30 days to undo this action."
}
```

### 2.4 Undo Delete Account

#### **POST /api/profile/undo-delete**

```
Response Body:
{
  "message": "Account deletion cancelled successfully"
}
```

## 3. Riot and Match Data Endpoints

### 3.1 Get Riot Match IDs by PUUID

#### **GET /api/riot/matches/{puuid}**

- Hits the live Riot API engine to pull historical match ID strings associated with a specific Player UUID.
- Query Params: count (optional, default=5)

```
Response Body:
{
  "NA1_51029301",
  "NA1_51028442",
  "NA1_51027119"
}
```

### 3.2 Get Single Player Filtered Match Summary

#### **GET /api/matches/{match\_id}/filtered**

- Resolves full match details directly through Riot's servers and outputs an optimized, lightweight runtime payload focused on a singular player's stats
- Query Parameters: puuid (string, required): The target player's exact PUUID.

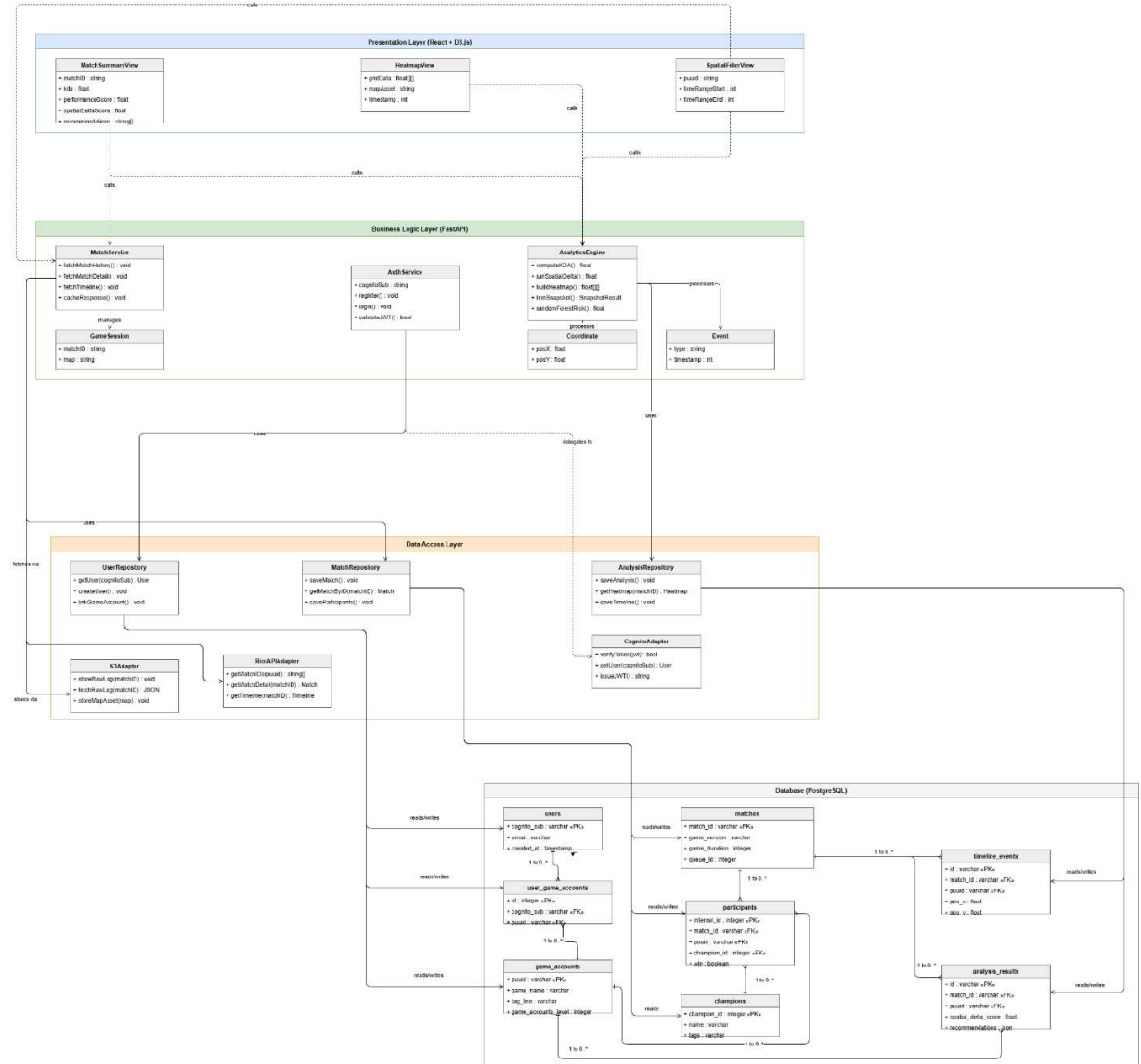
```
Response Body:
{
  "match_id": "NA1_51029301",
  "game_mode": "CLASSIC",
  "game_duration_seconds": 1930,
  "game_end_timestamp": "2026-05-13T14:22:00Z",
  "target_player": {
    "puuid": "0d2c7b5e-riot-puuid-example-123456",
    "riot_id": "Sn1per1#NA1",
    "champion_name": "Jinx",

```

```
    "champion_id": 222,  
    "team_position": "BOTTOM",  
    "win": true,  
    "kills": 12,  
    "deaths": 3,  
    "assists": 8,  
    "kda": 6.67,  
    "gold_earned": 15400,  
    "total_damage_dealt_to_champions": 28450,  
    "vision_score": 18,  
    "creep_score": {  
        "total_minions_killed": 235,  
        "neutral_minions_killed": 12,  
        "cs_per_minute": 7.7  
    },  
    "items": [  
        3031,  
        3046,  
        3006,  
        1055,  
        3072,  
        0,  
        3363  
    ]  
}
```

## Domain Model

### Complete System:



## Architectural Requirements

---

### Quality Requirements

#### Performance:

- The API and data processing system must process a standard match timeline log quickly and efficiently.

#### Scalability:

- The architecture must support concurrent data ingestion from multiple matches without degradation in processing speed.

#### Security:

- Data access must be restricted via JWT (JSON Web Tokens) to ensure team match data remains private.
- Role-based access is used to allow further control over what user can access and thereby increase security.

#### Maintainability:

- The backend should follow a modular "Core-and-Adapter" approach to allow for quick and easy updates when changes are made to the Match-V5 API.

### Architectural Patterns

- Layered Architecture Pattern

Vantage Point is built as an N-Tier system, consisting of the following layers:

- Front-end Layer: This mainly consists of the UI and basic functionality encoded within
- ML and Data Processing Layer: This layer contains the KNN model and is where all intensive data processing and analysis will be done.
- API and Backend Layer: The API will communicate with the Riot Match-V5 API outside of the system to get input data. Other backend processes are also contained in this layer.
- Database Layer: Here is where the system will store and organize all user and system data.

- Microservices Architecture Pattern

There is currently consideration to have the API built according to the microservices architecture pattern:

- API endpoints are to be group according to functionality.
- For example, endpoints for login and signup are grouped as a single service.
- Endpoints servicing the KNN model will be a service group.

- General game stats for basic processing and later display will be gotten from a dedicated group of endpoints.

## Design Patterns

- Composite Design Pattern

The match timeline data, as it is received from Riot's Match-V5 API as a string in approximate JSON format, will be translated into a composite object on the Data Processing Layer.

- This makes the data easier to manage and allows simpler access to different parts thereof.
- This also accounts for the differing shape of match timeline data.

- Observer Design Pattern

The observer will be set up to watch for input data from the API and trigger request and processing thereof at the appropriate data processing module

- This would theoretically allow for more streamlined match data processing and faster turnaround time.

## Constraints

- The system is almost entirely dependent on Riot's Match-V5 API for most of its functionality. This outside API has various restrictions and rate limits in place. This places these same limits on our system.
- The system is also reliant on the Account-V1 API for user information like puuid, which is necessary for obtaining the players match data.
- Another restriction caused by Riot's Match-V5 API is how much data can be collected in a set time frame for model training. Access to historic match data is limited to match data from Esports tournaments, which may not be enough to properly train the models for use by the average player.
- Map assets are only available as flat images, where data analysts will have to manually add in map contents (like obstacles, stationary goals, etc.).

## Technology Requirements

---

Vantage Point utilizes a modern, data-centric stack designed to handle high-frequency coordinate data and provide predictive insights into player behavior.

### Backend & Data Processing

- FastAPI (Python): Chosen for its high-performance asynchronous execution and native support for Pydantic. It handles API rate limiting and serves as the orchestration layer for ML inference.

- **Pandas & Scikit-learn:** These form the core of the analytics engine. Pandas is used for cleaning and normalizing game logs, while Scikit-learn implements K-Nearest Neighbors for snapshot comparison and Random Forest for calculating win-probability/risk metrics.
- **Static Analysis:** To maintain the type safety often missing in Python, the project enforces strict MyPy type checking and Ruff for high-speed linting.

#### Frontend & Visualization

- **React:** Provides a modular UI for the analysis dashboard, managing the state of match play back and filter controls.
- **D3.js:** The primary spatial engine. It maps game engine coordinates to screen pixels, ensuring that heatmaps and player paths are overlaid on map assets.
- **Tailwind CSS:** Used for a responsive, utility-first UI, ensuring the dashboard is usable across different screen resolutions.

#### Persistence & Infrastructure

- **PostgreSQL:** Stores relational data such as match histories, player profiles, and pre-calculated spatial benchmarks.
- **AWS (S3 & Lambda):** Utilized for serverless data fetching and cost-effective storage of large raw match log files.
- **Dev Containers:** To eliminate environment drift, the entire stack (including C-extensions for ML libraries) is containerized, ensuring consistency between local development and deployment.
- **AWS Cognito:** For secure user login, signup, and access control.